



PROYECTO FINAL

SISTEMA DE GESTIÓN DE TUTORÍAS PARTICULARES

(GRUPO 22)

Desarrollo Orientado a Objetos

Integrantes

Daniel Cristóbal Patricio López Ramírez (2025441799)

Eduardo Alexander Riveros Medina (MATRICULA)

Nicolás Fernando Silva Paredes (2024422430)

Durante el presente informe se aborda el proceso de diseño y desarrollo del Proyecto Final de Sistema de Gestión de Tutorías Particulares, decisiones tomadas e implementación de detalles lógicos para su correcto funcionamiento. El mismo, programado bajo el lenguaje Java y fue construido bajo los patrones de diseño de la Programación Orientada a Objetos en la medida que los integrantes del grupo 22 lo vimos necesario.

A continuación, se exponen las decisiones tomadas, los patrones de diseño implementados, las problemáticas afrontadas y un análisis autocrítico del trabajo grupal respecto al código y a la organización de los integrantes.

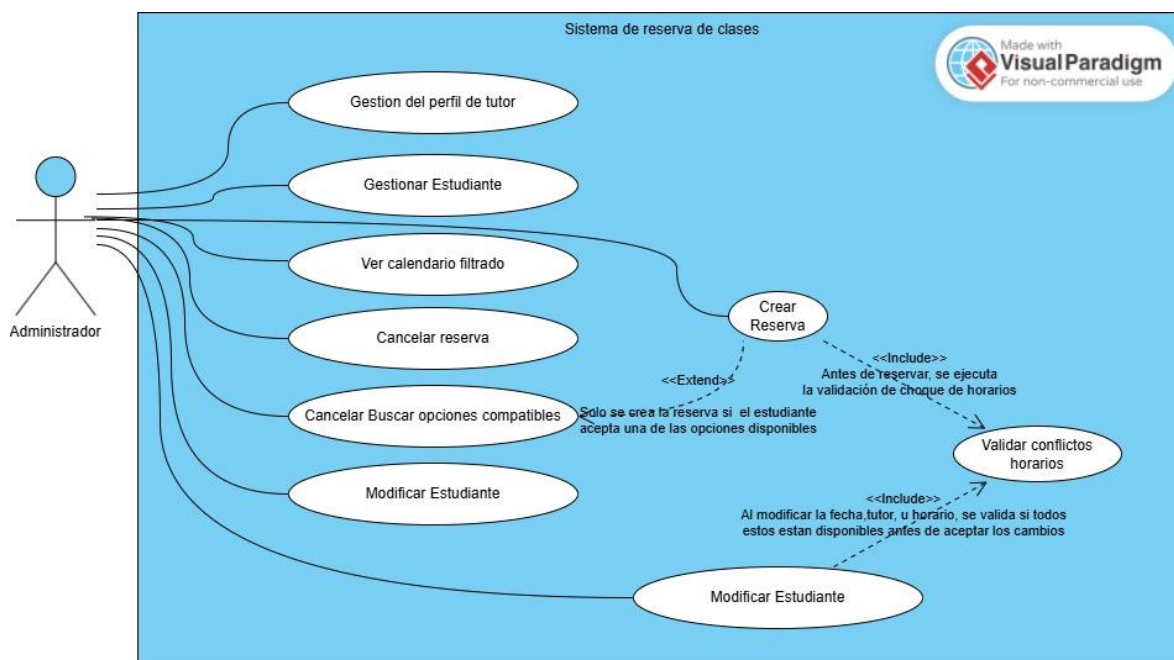


Figura 1. Diagrama de casos de uso

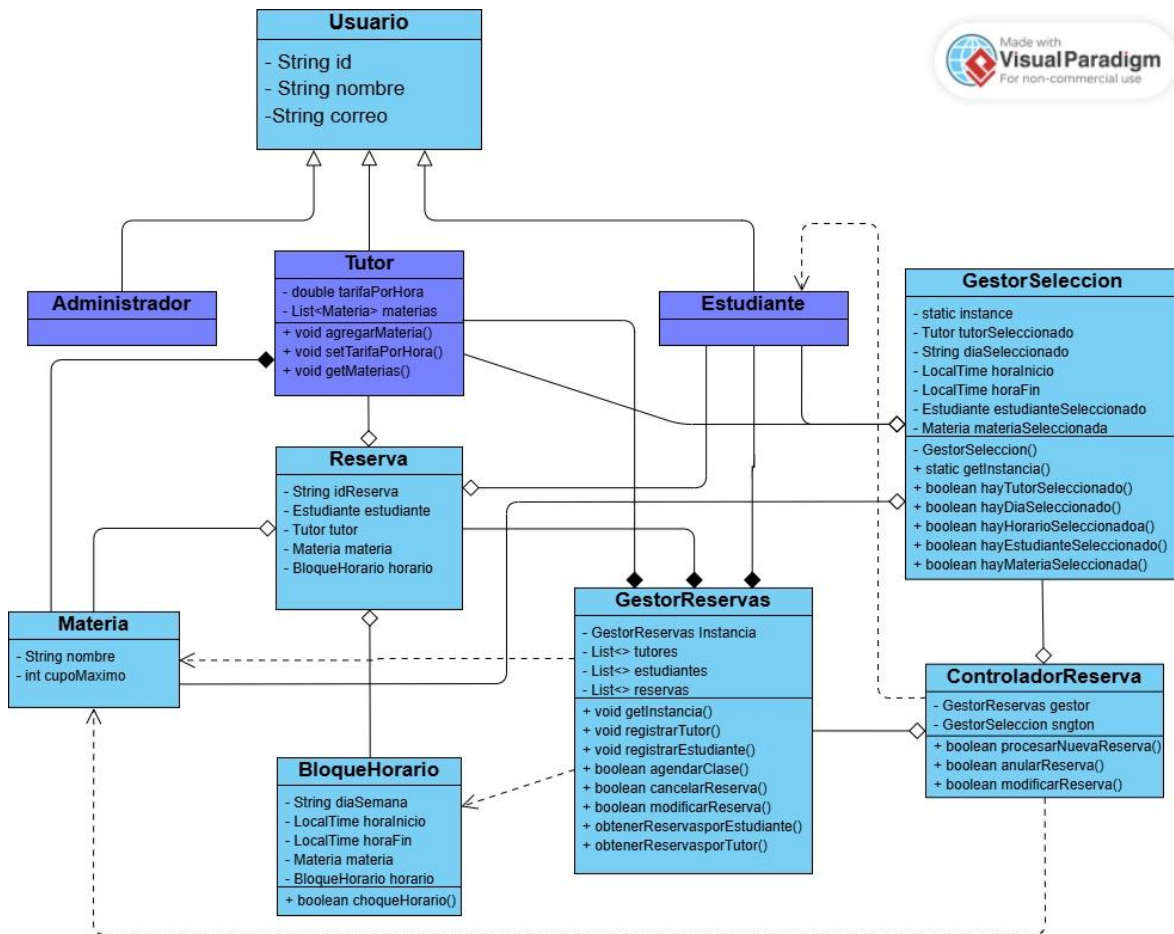


Figura 2. Diagrama UML de clases

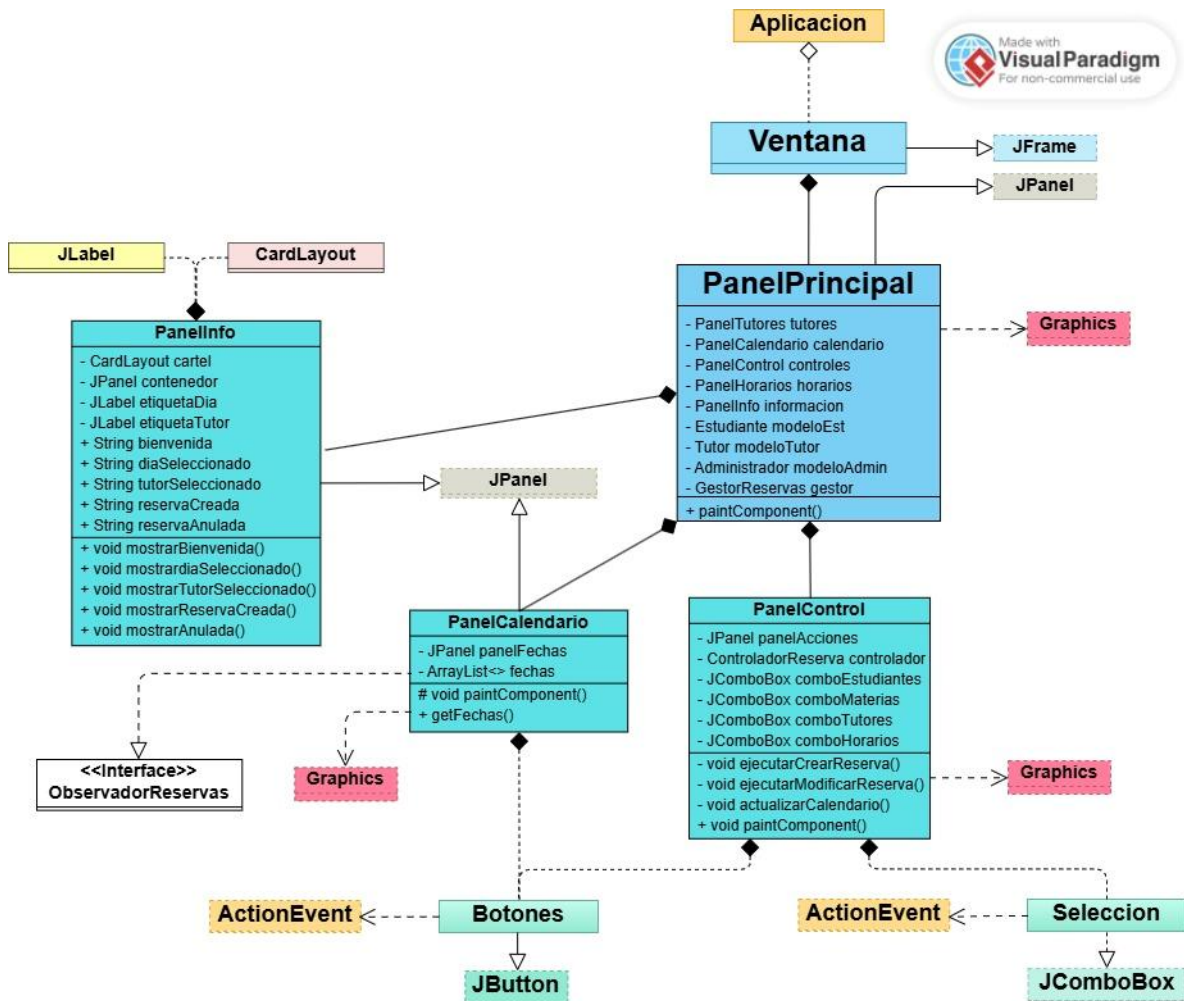


Figura 3. Diagrama UML de la Interfaz Gráfica

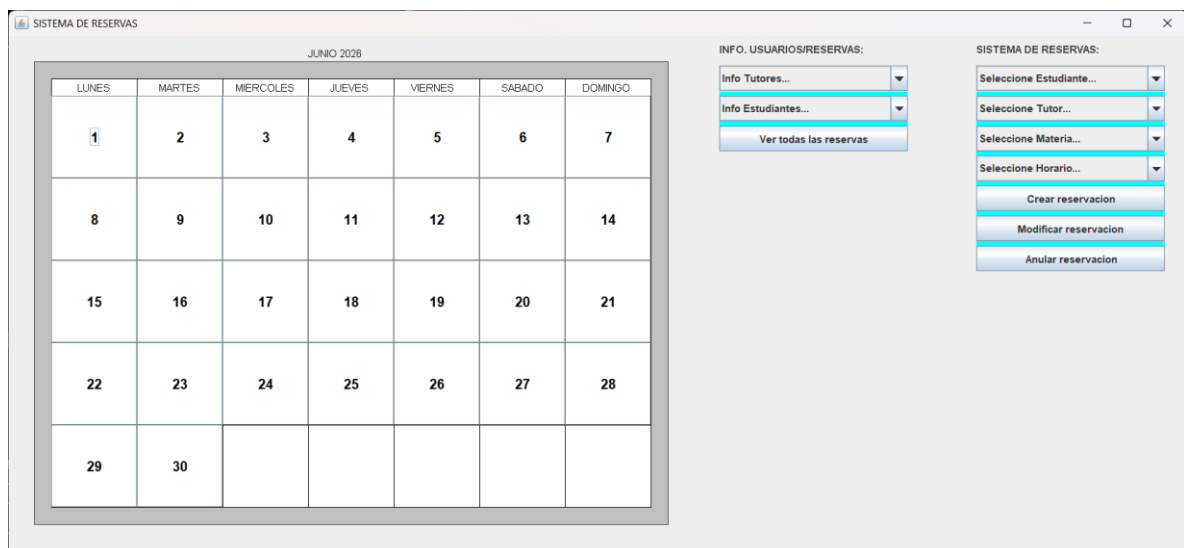


Figura 4. Muestra de la GUI

Decisiones en la Construcción del Código:

- Clase Materia: Inicialmente se tuvo la idea de tratarla como un string más asociado a otras listas que funcionarían como sus atributos, pero en lo que respecta a la programación orientada a objetos, se modificó y creó la clase Materia, la cual ya traía sus métodos y atributos correspondientes y se le trató como un objeto más dentro del código, simplificando su trabajo y uso con getters y setters para las revisiones de cupos máximos, alumnos inscritos y reservas de clases.

Patrones de Diseño:

- Factory: En un principio este fue uno de los patrones de diseño que se pensaba, serían de suma importancia, en las primeras fases de implementación. Luego se optó por mejores opciones, lo que termina en su descarte.
- Singleton: Se implementó principalmente en clases como *GestorReservas* y *GestorSelección*, ayudando a que el programa en general trabaje con las mismas listas de datos en vez de tener una lista por clase, por separado.
- Mediator: Se implementó en *ControladorReserva*, con el método *procesarNuevaReserva*, funcionando como un Holder de las selecciones del usuario en la GUI, ayudando a procesar posteriormente esos datos seleccionados.
- Observer: La implementación de un observador en esta aplicación fue esencial para la comunicación apropiada entre el *back-end* y el *front-end*, anterior a la implementación, ni la clase *GestorReservas* ni tampoco *PanelCalendario* contaban con una funcionalidad que brindara en todo momento, retroalimentación o recibiera esta, para mantener a la interfaz actualizada en todo momento.

Problemáticas afrontadas:

Al momento de hacer clic en la selección del profesor para ver su disponibilidad horaria o agendar una clase con él, se perdía la opción seleccionada al cambiar el menú, así que con la clase Gestor Selección y el método *procesarNuevaReserva*, se logró solventar el problema.

También, hubo problemas en las revisiones de los choques horarios al modificarlos, ya que el bloque horario que se quería modificar seguía contando como horario ocupado, por ende, en la clase *GestorReserva*, el método modificar Reserva, se modificó para que, al intentar modificar una reserva, se ignorara el ID de la que se estaba intentando cambiar.

Análisis Crítico y Puntos Débiles del Código:

Un gran problema que podemos identificar como grupo dentro del código general del programa, es que la clase *GestorReservas* tiene demasiadas responsabilidades, ya que guarda los datos, revisa los cupos máximos por materia, verifica choques horarios y registra usuarios. Más adelante, si lográramos identificar la existencia de un error, nos costaría más encontrar la fuente de ese error, por ende, en un futuro podríamos diversificar esas responsabilidades, haciendo que el código sea más limpio y fácil de mantener a largo plazo.